Practical No.1

Create a C# console application to demonstrate variables, data types and operators.

Code:

```
using System;
namespace BasicConceptsDemo {
   class Program {
      static void Main(string[] args) {
         // 1. DATA TYPES AND VARIABLES
         // Integer types
         int age = 25;
         long largeNumber = 9000000000L;
         // Floating-point types
         double pi = 3.14159;
         decimal price = 19.99m; // High precision for financial use
         // Character and String types
         char grade = 'A';
         string message = "Hello, C# World!";
         // Boolean type
         bool isProgrammingFun = true;
         // 2. OPERATORS
         // Arithmetic Operators
         int a = 10, b = 3;
         int sum = a + b;         // 13
         int difference = a - b;    // 7
         int product = a * b;       // 30
         int quotient = a / b;      // 3 (integer division)
         int remainder = a % b;     // 1
         // Relational (Comparison) Operators
         bool isGreater = a > b;    // true
         bool isEqual = (a == b);   // false
        // Logical Operators
```

```
bool logicResult = (a > 5 && b < 5); // true (AND)
// 3. OUTPUT
Console.WriteLine("--- Variables and Data Types ---");
Console.WriteLine($"String: {message}");
Console.WriteLine($"Integer: {age}");
Console.WriteLine($"Decimal Price: ${price}");
Console.WriteLine($"Boolean: {isProgrammingFun}");
Console.WriteLine("\n--- Operator Results ---");
Console.WriteLine($"{a} + {b} = {sum}");
Console.WriteLine($"{a} divided by {b} has a remainder of {remainder}");
Console.WriteLine($"Is {a} greater than {b}? {isGreater}");
Console.WriteLine($"Logical AND result: {logicResult}");
// Keep console window open
Console.WriteLine("\nPress any key to exit...");
Console.ReadKey(); } } }
```

Output:

```
--- Variables and Data Types ---
String: Hello, C# World!
Integer: 25
Decimal Price: $19.99
Boolean: True

--- Operator Results ---
10 + 3 = 13
10 divided by 3 has a remainder of 1
Is 10 greater than 3? True
Logical AND result: True
```

Practical No.2

Write a C# program using loops and conditional logic.

Program 1 Code:

```
using System;
class Program {
   static void Main() {
      Console.WriteLine("--- Number Classifier ---");
      // Loop: Iterates from 1 to 10
      for (int i = 1; i <= 10; i++) {
         // Conditional Logic: Checks if the current number is even or odd
         if (i % 2 == 0) {
            Console.WriteLine($"{i} is an EVEN number."); }
         else {
            Console.WriteLine($"{i} is an ODD number."); } }
      Console.WriteLine("-------------------------");
      Console.WriteLine("Loop finished. Press any key to exit.");
      Console.ReadKey(); } }
```

Output:

```
--- Number Classifier ---
1 is an ODD number.
2 is an EVEN number.
3 is an ODD number.
4 is an EVEN number.
5 is an ODD number.
6 is an EVEN number.
7 is an ODD number.
8 is an EVEN number.
9 is an ODD number.
10 is an EVEN number.
-------------------------
```

Program 2 Code:

```
using System;
namespace Loop {
	class DoWhileLoop {
		public static void Main(string[] args) {
			int i = 1, n = 5, product;
			do {
				product = n * i;
				Console.WriteLine("{0} * {1} = {2}", n, i, product);
				i++;
			} while (i <= 10); } } }
```

Output:

```
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50
```

Practical No.3

Design a class in C# with attributes and methods using inheritance.

Code:

```
using System;
// Base Class (Parent)
public class Vehicle {
   // Attributes
   public string Brand = "Ford";
   // Method
   public void Honk()  {
      Console.WriteLine("Tuut, tuut!"); } }
// Derived Class (Child)
// The ":" symbol denotes inheritance
public class Car : Vehicle {
   // Additional Attribute unique to Car
   public string ModelName = "Mustang";
   // Additional Method unique to Car
   public void DisplayInfo() {
      // Accesses Brand from Vehicle and ModelName from Car
      Console.WriteLine($"Brand: {Brand}, Model: {ModelName}"); } }
class Program {
   static void Main(string[] args) {
      // Create a Car object
      Car myCar = new Car();
      // Call inherited method from Vehicle class
      myCar.Honk();
      // Call method from Car class
      myCar.DisplayInfo(); } }
```

Output:

```
Tuut, tuut!
Brand: Ford, Model: Mustang
```

Practical No. 4

Factory Design Pattern in C#

Code: CreditCard.cs

```
namespace FactoryDesignPattern {
publicinterfaceCreditCard{
stringGetCardType();
intGetCreditLimit();
intGetAnnualCharge();}}
```

Code: MoneyBack.cs

```
namespace FactoryDesignPattern{
classMoneyBack :CreditCard{
publicstringGetCardType(){
return"MoneyBack";}
publicintGetCreditLimit(){
return15000;}
publicintGetAnnualCharge(){
return500;}}}
```

Code: Titanium.cs:

```
namespace FactoryDesignPattern{
publicclassTitanium :CreditCard{
publicstringGetCardType(){
return"Titanium Edge";}
publicintGetCreditLimit(){
return25000;}
publicintGetAnnualCharge(){
return1500;}}}
```

Code: Platinum.cs:

```
namespace FactoryDesignPattern{
publicclassPlatinum :CreditCard{
publicstringGetCardType(){
return"Platinum Plus";}
publicintGetCreditLimit(){
return35000;}
publicintGetAnnualCharge(){
return2000;}}}
```

Code: Program.cs

```
using System;
namespace FactoryDesignPattern{
class Program{
staticvoidMain(string[]args){
//Generally we will get the Card Type from UI.
//Here we are hardcoded the card type
stringcardType = "MoneyBack";
CreditCardcardDetails = null;
//Based of the CreditCard Type we are creating the
//appropriate type instance using if else condition
if(cardType == "MoneyBack"){
cardDetails = newMoneyBack();}
elseif(cardType == "Titanium"){
cardDetails = newTitanium();}
elseif(cardType == "Platinum"){
cardDetails = newPlatinum();}
if(cardDetails != null){
Console.WriteLine("CardType : " + cardDetails.GetCardType());
Console.WriteLine("CreditLimit : " + cardDetails.GetCreditLimit());
```

```
Console.WriteLine("AnnualCharge :" + cardDetails.GetAnnualCharge());}
else{
Console.Write("Invalid Card Type");}
Console.ReadLine();}}}
```

Output:

```
CardType : MoneyBack
CreditLimit : 15000
AnnualCharge :500
```

Code: CreditCardfactory.cs

```
namespace FactoryDesignPattern{
publicclassCreditCardFactory{
publicstaticCreditCardGetCreditCard(stringcardType){
CreditCardcardDetails = null;
if(cardType == "MoneyBack"){
cardDetails = newMoneyBack();}
elseif(cardType == "Titanium"){
cardDetails = newTitanium();}
elseif(cardType == "Platinum"){
cardDetails = newPlatinum();}
returncardDetails;}}}
```

Code: Program.cs

```
using System;
namespace FactoryDesignPattern{
class Program{
staticvoidMain(string[]args){
CreditCardcardDetails = CreditCardFactory.GetCreditCard("Platinum");
if(cardDetails != null){
Console.WriteLine("CardType : " + cardDetails.GetCardType());
```

```
Console.WriteLine("CreditLimit : " + cardDetails.GetCreditLimit());

Console.WriteLine("AnnualCharge :" + cardDetails.GetAnnualCharge());}

else{

Console.Write("Invalid Card Type");}

Console.ReadLine();}}}
```

Output:

```
CardType : Platinum Plus
CreditLimit : 35000
AnnualCharge :2000
```

Practical No. 5

Create a singleton class in C#

<u>Code:</u> Singleton Class Implementation

```
using System;
// Define the Singleton class as sealed to prevent inheritance
public sealed class Singleton{
   // Private static readonly instance
   private static readonly Singleton instance = new Singleton();
   // Explicit static constructor to prevent beforefieldinit
   static Singleton(){ }
   // Private constructor prevents external instantiation
   private Singleton() {
      Console.WriteLine("Singleton instance created."); }
   // Public static property provides global access point
   public static Singleton Instance {
      get{
         return instance;}}
   // Example method to demonstrate the singleton is working
   public void ShowMessage(string msg){
      Console.WriteLine("Message from Singleton: " + msg);}}
// Client code
class Program{
   static void Main(string[] args){
      Singleton s1 = Singleton.Instance;
      Singleton s2 = Singleton.Instance;
      if (s1 == s2){
         Console.WriteLine("Singleton works, both variables contain the same instance.");}
      else{
         Console.WriteLine("Singleton failed, variables contain different instances.");}
      s1.ShowMessage("Hello World!");
```

```
Console.ReadLine(); // Keeps console window open in VS2012}}
```

Output :

Practical No.6

Implementing repository pattern in C#

Step 1. Create Database

Let's create a simple database (BasicDb) containing a single table with the name of Product_Table. The Backup and Script file is attached simply download it and restore it in SQL. You can choose any one file for restoration.

Step 2. Setup ASP.NET MVC Project

Select MVC

Step 3. Add Entity Framework

Create an Entity Framework to give the model name DataContext.

Select Entity Framework Designer from the database.

Select Database

Change the name to (DataContext) it is optional you can keep it as it is.

Select Table

Step 4. Create DAL Folder

Create a Folder in the model folder with the name (DAL) where we will implement our repository.

Step 5. Define Interface

Code: IProductRepository.cs

```
using System;

using System.Collections.Generic;

using System.Linq;

using System.Text;

using System.Threading.Tasks;

namespace Repositiory_Pattern.Models.DAL{

   interface IProductRepository{

      IEnumerable<Product_Table> GetProducts();

      Product_Table GetProductById(int ProductId);

      void InsertProduct(Product_Table product_);

      void UpdateProduct(Product_Table product_);

  void DeleteProduct(int ProductId);
```

```
    void SaveChanges();}}
```

Step 6. Implement Repository Class

Create a class in the DAL folder (ProductRepository.cs) This will be a child class and will implement all the methods of the (IProductRepository.cs) class.

Code: ProductRepository.cs

```
using System;

using System.Collections.Generic;

using System.Linq;

using System.Web;

namespace Repositiory_Pattern.Models.DAL{

  public class ProductRepository : IProductRepository{

    private DataContext _dataContext;

    public ProductRepository(DataContext dataContext){

      this._dataContext = dataContext;}

    public void DeleteProduct(int ProductId){

      Product_Table product_ = _dataContext.Product_Table.Find(ProductId);

      _dataContext.Product_Table.Remove(product_);}

    public Product_Table GetProductById(int ProductId){

      return _dataContext.Product_Table.Find(ProductId);}

    public IEnumerable<Product_Table> GetProducts(){

      return _dataContext.Product_Table.ToList();}

    public void InsertProduct(Product_Table product_){

      _dataContext.Product_Table.Add(product_);}

    public void SaveChanges(){

      _dataContext.SaveChanges();}

    public void UpdateProduct(Product_Table product_){

      _dataContext.Entry(product_).State = System.Data.Entity.EntityState.Modified;}}}
```

Step 7. Modify Home Controller

Now just go to our built-in Home Controller and paste this code.

Code: HomeController.cs

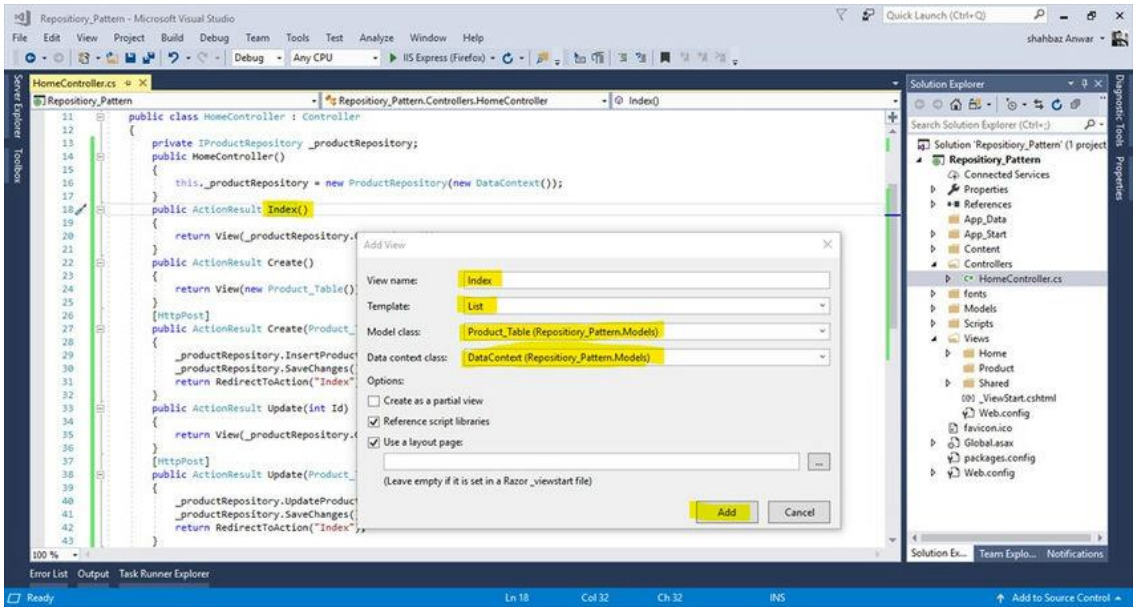
```
using Repositiory_Pattern.Models;
using Repositiory_Pattern.Models.DAL;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
namespace Repositiory_Pattern.Controllers{
    public class HomeController : Controller{
        private IProductRepository _productRepository;
        public HomeController(){
            this._productRepository = new ProductRepository(new DataContext());}
        public ActionResult Index(){
            return View(_productRepository.GetProducts());}
        public ActionResult Create(){
            return View(new Product_Table());}
        [HttpPost]
        public ActionResult Create(Product_Table product){
            _productRepository.InsertProduct(product);
            _productRepository.SaveChanges();
            return RedirectToAction("Index"); }
        public ActionResult Update(int Id) {
            return View(_productRepository.GetProductById(Id));}
        [HttpPost]
        public ActionResult Update(Product_Table product){
            _productRepository.UpdateProduct(product);
            _productRepository.SaveChanges();
            return RedirectToAction("Index");}
        public ActionResult Delete(int Id){
            _productRepository.DeleteProduct(Id);
```

```
_productRepository.SaveChanges();
return RedirectToAction("Index");}}}
```

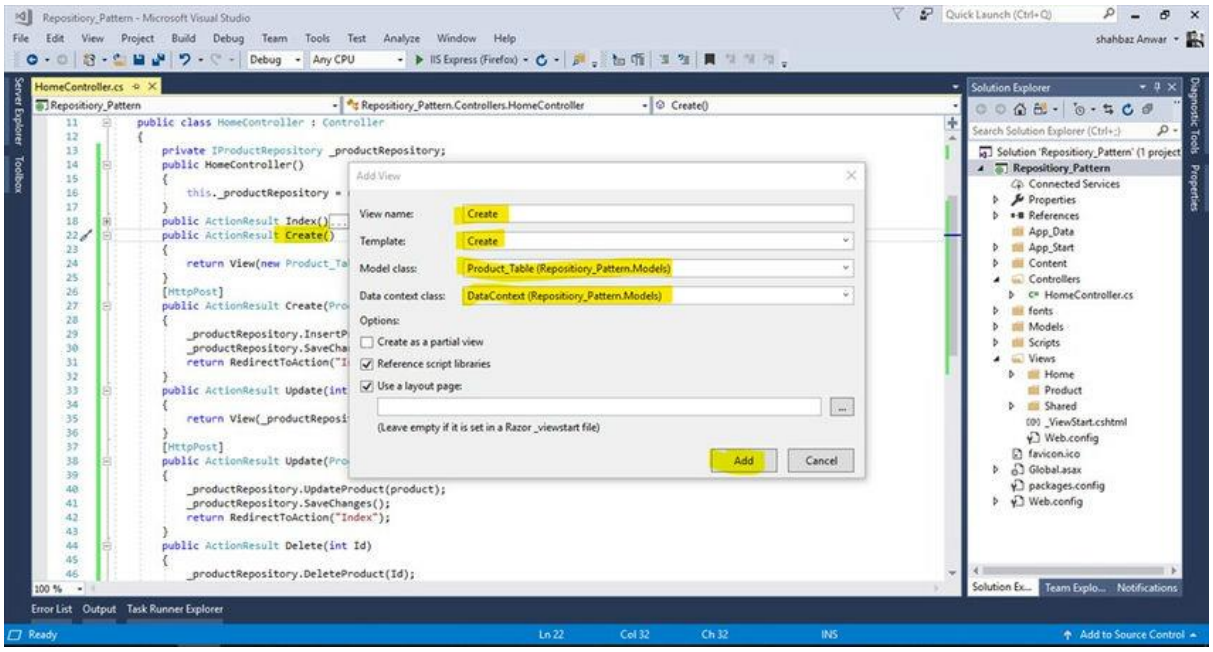
Step 8. Create Views

Now create the views Index, Create and Update. Just right-click on the method name.

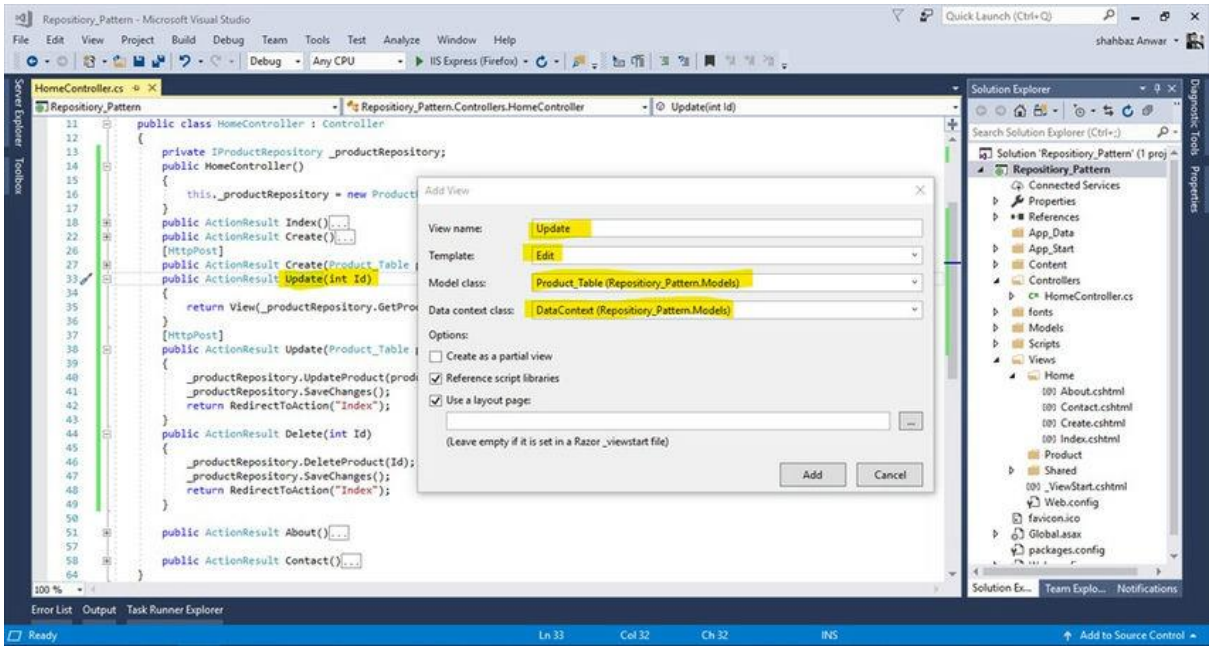
Index.cshtml



Create.cshtml

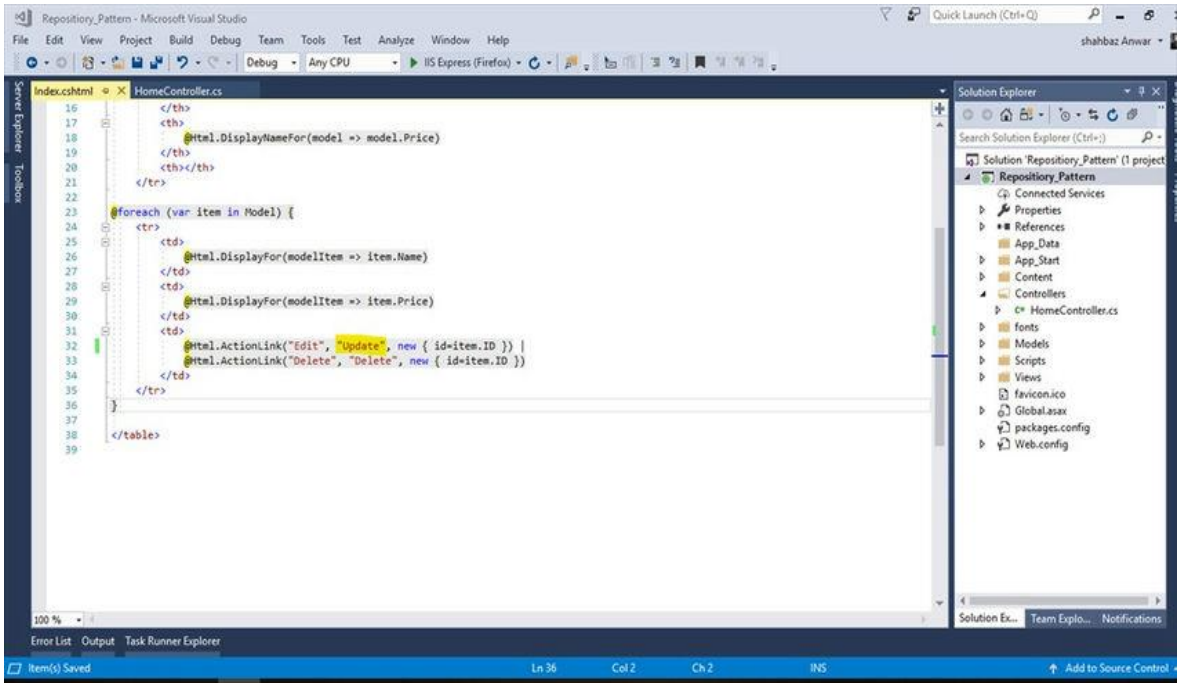


Update.cshtml



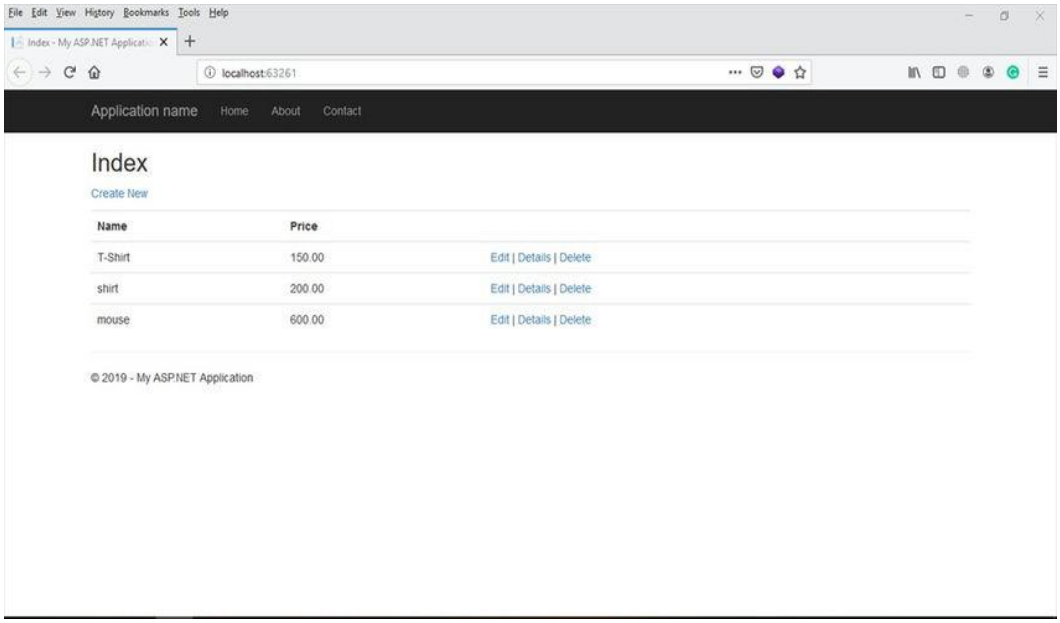
Step 9. Adjust Index View

Open the Index view and change the action name Edit to Update and save.



Step 10. Run and Test

Now just run and see the result in the browser.


File Edit View History Bookmarks Tools Help
Index - My ASP.NET Applicatio
localhost:63261
Application name
Home
About
Contact
Index
Create New
Name
Price
T-Shirt
150.00
Edit | Details | Delete
shirt
200.00
Edit | Details | Delete
mouse
600.00
Edit | Details | Delete
© 2019 - My ASP.NET Application

Practical No. 7

Create a “Hello World” application with ASP.NET Core MVC.

<u>Code:</u> HomeController.cs

```
public ActionResult HelloWorld()
{
ViewBag.Message = "Hello World!";
   return View();
}
```

Html code –

```
@{
ViewBag.Title = "Hello World";
}

<h2>@ViewBag.Message</h2>
```

<u>Output :</u>

Press F5 to build and run your application. Visual Studio will launch a web browser displaying your "Hello World!" message.

Practical No. 8 & 12

Create crud application using Entity Framework and AJAX

STEP 1: Create ASP.NET MVC Project

1. Open **Visual Studio 2012**
2. Click **File → New → Project**
3. Select:
    - **Visual C# → Web**
    - Choose **ASP.NET MVC 4 Web Application**
4. Name it: MvcCrudAjaxDemo
5. Select:
    - Template: **Internet Application**
    - View Engine: **Razor**
6. Click **OK**

STEP 2: Install Entity Framework (if not already installed)

Visual Studio 2012 usually includes EF 5.

If needed:

1. Open **Tools → Library Package Manager → Package Manager Console**
2. Run:

Install-Package EntityFramework

STEP 3: Create Model Class

Right-click **Models → Add → Class**

Create Student.cs

```
using System.ComponentModel.DataAnnotations;
namespace MvcCrudAjaxDemo.Models{
   public class Student {
      public int Id { get; set; }
      [Required]
      public string Name { get; set; }
      public int Age { get; set; }
      public string City { get; set; }}}
```

STEP 4: Create DbContext

Add new class: StudentContext.cs

```
using System.Data.Entity;
namespace MvcCrudAjaxDemo.Models{
   public class StudentContext : DbContext{
      public DbSet<Student> Students { get; set; }}}
```

STEP 5: Add Connection String

Open **Web.config**

Add inside <connectionStrings>:

```
<connectionStrings>
  <add name="StudentContext"
      connectionString="Data Source=.;Initial Catalog=StudentDB;Integrated Security=True"
      providerName="System.Data.SqlClient" />
</connectionStrings>
```

Now when you run the project, EF will create the database automatically.

STEP 6: Create Controller

Right-click **Controllers → Add → Controller**

Choose:

- **MVC 5 Controller with views, using Entity Framework**
- Model class: Student
- Data context class: StudentContext

Click **Add**

This automatically creates CRUD actions.

STEP 7: Modify Controller for AJA

Open StudentController.cs

Replace Create POST action with:

```
[HttpPost]
public JsonResult Create(Student student){
   if (ModelState.IsValid){
      db.Students.Add(student);
      db.SaveChanges();
      return Json(new { success = true });}
   return Json(new { success = false });}
```

STEP 8: Modify Index View for AJAX

Open:

Views → Student → Index.cshtml

Replace content with:

```
@model IEnumerable<MvcCrudAjaxDemo.Models.Student>

<h2>Student List</h2>
<button id="btnAdd">Add Student</button>
<div id="studentForm" style="display:none;">
   <input type="hidden" id="Id" />
   <input type="text" id="Name" placeholder="Name" />
   <input type="number" id="Age" placeholder="Age" />
   <input type="text" id="City" placeholder="City" />
   <button id="btnSave">Save</button>
</div>
<table border="1">
   <thead>
      <tr>
         <th>Name</th>
         <th>Age</th>
         <th>City</th>
      </tr>
   </thead>
   <tbody id="studentTable">
      @foreach (var item in Model){
         <tr>
            <td>@item.Name</td>
            <td>@item.Age</td>
            <td>@item.City</td>
         </tr>}
   </tbody>
</table>


@section Scripts {
<script src="~/Scripts/jquery-1.8.2.min.js"></script>
<script>
   $("#btnAdd").click(function () {
      $("#studentForm").show();});
$("#btnSave").click(function () {
      var student = {
         Name: $("#Name").val(),
         Age: $("#Age").val(),
         City: $("#City").val() };
      $.ajax({
         url: '@Url.Action("Create", "Student")',
         type: 'POST',
         data: student,
         success: function (result) {
            if (result.success) {
               alert("Saved Successfully");
               location.reload();}}});});</script>}
```

STEP 9: Add AJAX Delete Method (Optional)

In Controller:

```
[HttpPost]
public JsonResult Delete(int id){
   Student student = db.Students.Find(id);
   db.Students.Remove(student);
   db.SaveChanges();
   return Json(new { success = true });}
```

Add delete button in table and call AJAX similarly.

STEP 10: Run Application

Press **F5**

- Database will be created automatically.
- You can:
    - Create student
    - View student
    - Delete student (if implemented)

Practical No.10

To design an ASP.NET project for cookies and sessions in Visual Studio 2012

You will use **Web Forms** and C# code-behind files. Cookies store data on the client side, while sessions store data on the server side, typically using a session ID stored in a temporary cookie.

Part 1: Project Setup

1. **Open Visual Studio 2012** and select **File** > **New Project**.
2. In the New Project window, expand **Installed** > **Templates** > **Visual C#** > **Web**.
3. Select **ASP.NET Empty Web Application** (or **ASP.NET Web Forms Application**) and provide a name and location, then click **OK**.
4. In the **Solution Explorer**, right-click your project name and select **Add** > **New Item** > **Web Form**. Name it Default.aspx and click **Add**.
5. Add another Web Form named NextPage.aspx.

Part 2: Implementing Cookies

Cookies are stored on the client's browser and can persist across sessions if an expiration time is set.

Writing a Cookie (Default.aspx.cs)

1. Add a TextBox (ID: txtName) and a Button (ID: btnCreateCookie) to the Default.aspx page's HTML design.
2. In the Default.aspx.cs code-behind file, add a click event handler for the button:

```
protected void btnCreateCookie_Click(object sender, EventArgs e){
  // Create a new cookie object and set its name
  HttpCookie userCookie = new HttpCookie("UserInfo");
  // Store values in the cookie using key-value pairs
  userCookie["Name"] = txtName.Text;
  userCookie["LoginTime"] = DateTime.Now.ToString();
  // Set the cookie to expire in 1 day (makes it persistent)
  userCookie.Expires = DateTime.Now.AddDays(1);
  // Add the cookie to the response to send it to the client's browser
  Response.Cookies.Add(userCookie);

  // Optional: Redirect to another page to demonstrate persistence
```

```
  Response.Redirect("NextPage.aspx");}
```

Reading a Cookie (NextPage.aspx.cs)

1. Add a Label (ID: lblName) to the NextPage.aspx page's HTML design.
2. In the NextPage.aspx.cs code-behind file, read the cookie in the Page_Load event:

```
protected void Page_Load(object sender, EventArgs e){
  // Retrieve the cookie from the request
  HttpCookie reqCookies = Request.Cookies["UserInfo"];
  if (reqCookies != null){
    // Check if the cookie exists before accessing its values
    string userName = reqCookies["Name"];
    string loginTime = reqCookies["LoginTime"];
    lblName.Text = $"Welcome back, {userName}! Last visit: {loginTime}";}
  else{
    lblName.Text = "No cookie found or cookie expired.";   }}
```

Part 3: Implementing Sessions

Session data is stored on the server and is tied to a unique session ID, usually managed by a non-persistent cookie by default.

Writing a Session Variable (Default.aspx.cs)

1. Use the same Default.aspx as before. In the btnCreateCookie_Click method, you can add session logic:

```
protected void btnCreateCookie_Click(object sender, EventArgs e){
  // ... (cookie code as above) ...
  // Set a session variable
  Session["UserId"] = txtName.Text;
  Response.Redirect("NextPage.aspx");}
```

Reading a Session Variable (NextPage.aspx.cs)

1. In the NextPage.aspx.cs code-behind file, read the session variable in the Page_Load event:

```
protected void Page_Load(object sender, EventArgs e){
  // ... (cookie code as above) ...
  // Read a session variable
  if (Session["UserId"] != null) {
```

```
    string userId = Session["UserId"].ToString();
    // You can use this data for further application logic  }
  else  {
    // Session expired or not set }}
```

Part 4: Configuration (Web.config)

You can configure session behavior in the Web.config file.

1. Open Web.config in your Solution Explorer.
2. Add the <sessionState> element within the <system.web> section:

```xml
<system.web>
  <!-- Session state configuration -->
  <!-- mode="InProc" (default, stores in web server memory) -->
  <!-- timeout="20" (default, in minutes) -->
  <sessionState mode="InProc" timeout="1" />


  <!-- Other system.web elements like <compilation> or <pages> might be here -->
</system.web>
```

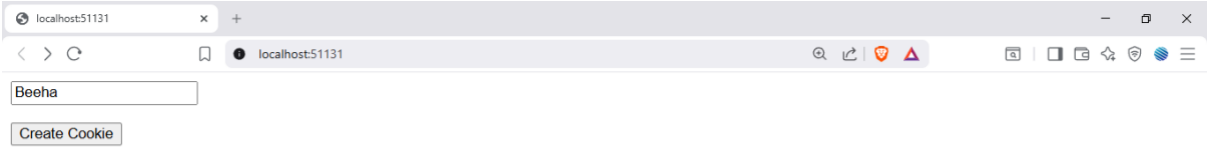
This example sets the session timeout to 1 minute for testing purposes. After 1 minute of inactivity, the Session["UserId"] would be null.

Key Points

- **Cookies** are client-side, used for persistence and preferences.
- **Sessions** are server-side, used for managing user state during an active visit (e.g., login status, shopping cart).
- The Response.Cookies.Add() method sends a cookie to the client, and Request.Cookies[] retrieves it.
- The Session[] object stores and retrieves server-side session data.

It is a best practice to check for null when retrieving cookies or session variables to prevent errors.

Output:


localhost:51131
localhost:51131
Beeha
Create Cookie

localhost:51131/NextPage.aspx
localhost:51131/NextPage.aspx
Welcome back, Beeha! Last visit: 23-02-2026 03:00:21

Practical No.11

Form Authentication

Step 1: Open Visual Studio then go to the File Menu where we click New, then Project and select the ASP.NET web application and assign the name of the application in pop up menu.
Step 2: After selecting the web application select an empty template here.
Step 3: In our web application here we add two pages one login.aspx and another welcome.aspx.

Step 4: Now we set web.config file to implement the authentication of web application.

```
<configuration>
  <appSettings>
    <add key="ValidationSettings:UnobtrusiveValidationMode" value="None" />
  </appSettings>
  <system.web>
    <compilation debug="true" targetFramework="4.5" />
    <authentication mode="Forms">
      <forms loginUrl="login.aspx" defaultUrl="welcome.aspx">
        <credentials passwordFormat="Clear">
          <user name="abhishek" password="abhi@123"/>
          <user name="Kantesh" password="sinha@123" />
        </credentials>
      </forms>
    </authentication>
    <authorization>
      <deny users="?"/>
    </authorization>
    <httpRuntime targetFramework="4.5" />
  </system.web>
</configuration>
```

Steps 5: Now we create a login form in *login.aspx* page. For that we will write the code for login form here,

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Login.aspx.cs" Inherits="Form Auth.Login" %>

<!DOCTYPE html>
```

```
<html
   xmlns="http://www.w3.org/1999/xhtml">
   <head runat="server">
      <title></title>
   </head>
   <body>
      <form id="form1" runat="server">
         <h3>  Login Page</h3>
         <table>
            <tr>
               <td>   UserName:</td>
               <td>  <asp:TextBox ID="UserName" runat="server" />  </td>
               <td> <asp:RequiredFieldValidator ID="RequiredFieldValidator1"
         ControlToValidate="UserName"
         Display="Dynamic"
         ErrorMessage="Cannot be empty."
         runat="server" />  </td>
            </tr>
            <tr>
               <td>  Password:</td>
               <td>  <asp:TextBox ID="UserPass" TextMode="Password"
          runat="server" />  </td>
               <td>  <asp:RequiredFieldValidator ID="RequiredFieldValidator2"
         ControlToValidate="UserPass"
         ErrorMessage="Cannot be empty."
         runat="server" />  </td>
            </tr>
            <tr>
               <td>  Remember me?</td>
               <td>  <asp:CheckBox ID="chkboxPersist" runat="server" />   </td>
            </tr>
```

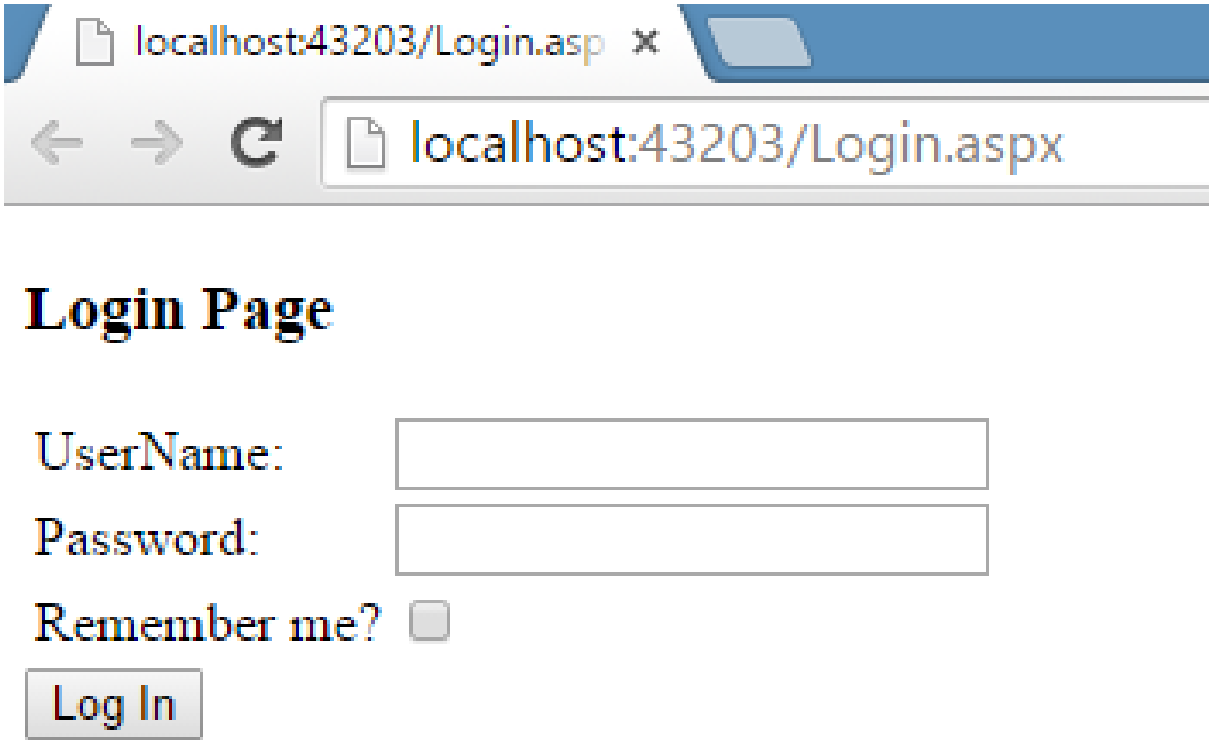
```
            </table>
            <asp:Button ID="Submit1" OnClick="Login_Click" Text="Log In"
        runat="server" />
            <p>  <asp:Label ID="Msg" ForeColor="red" runat="server" />  </p>
         </form>
       </body>
     </html>
```

Step 6: Here we write the code for Login button in *login.aspx.cs* page.

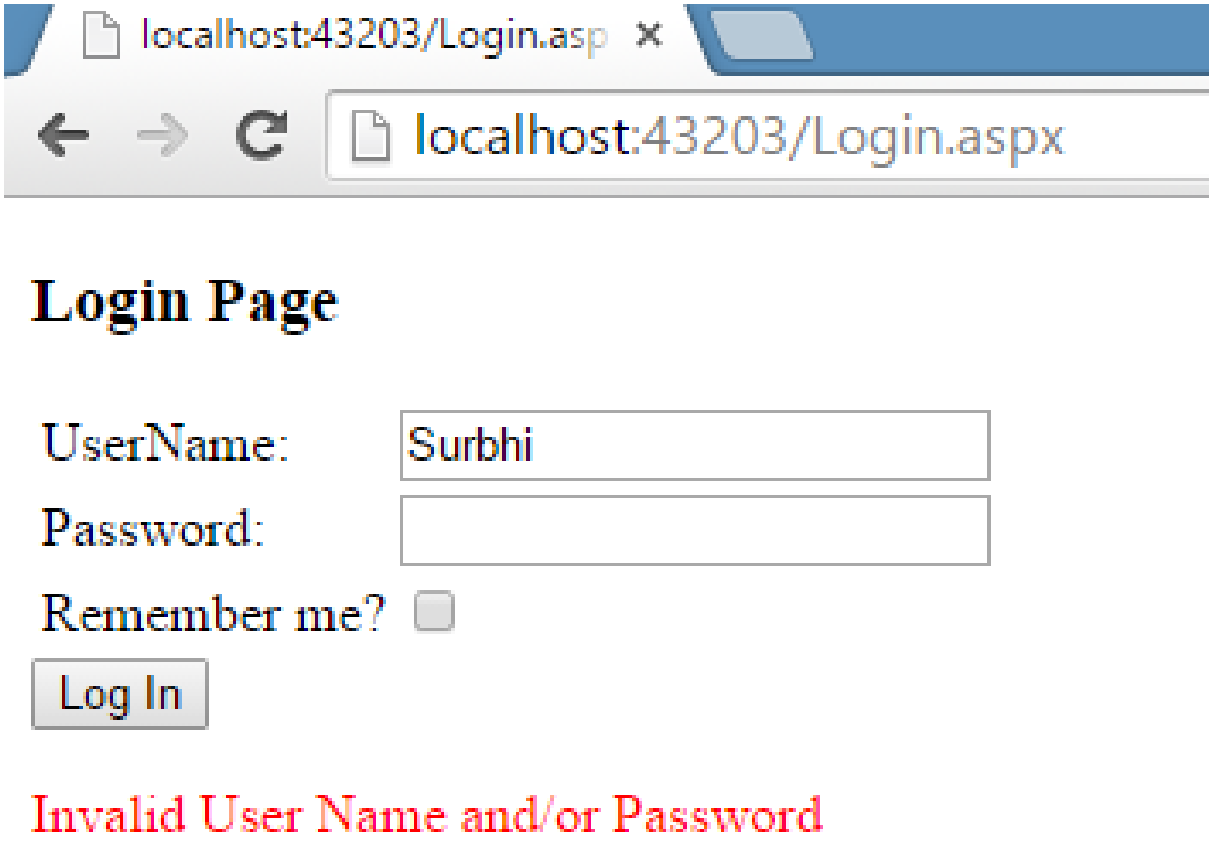
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
using System.Web.Security;

namespace FormAuth {
   public partial class Login: System.Web.UI.Page {
      protected void Page_Load(object sender, EventArgs e) {  }
      protected void Login_Click(object sender, EventArgs e) {
         if (FormsAuthentication.Authenticate(UserName.Text, UserPass.Text)) {
            FormsAuthentication.RedirectFromLoginPage(UserName.Text, chkboxPersist.Checked);
         } else {
            Msg.Text = "Invalid User Name and/or Password";  }}}}
```

Step 7: After writing the code of these pages now we are ready to execute the web application. To run the application we click Ctrl+F5 . After debugging the application our login page open in the browser.

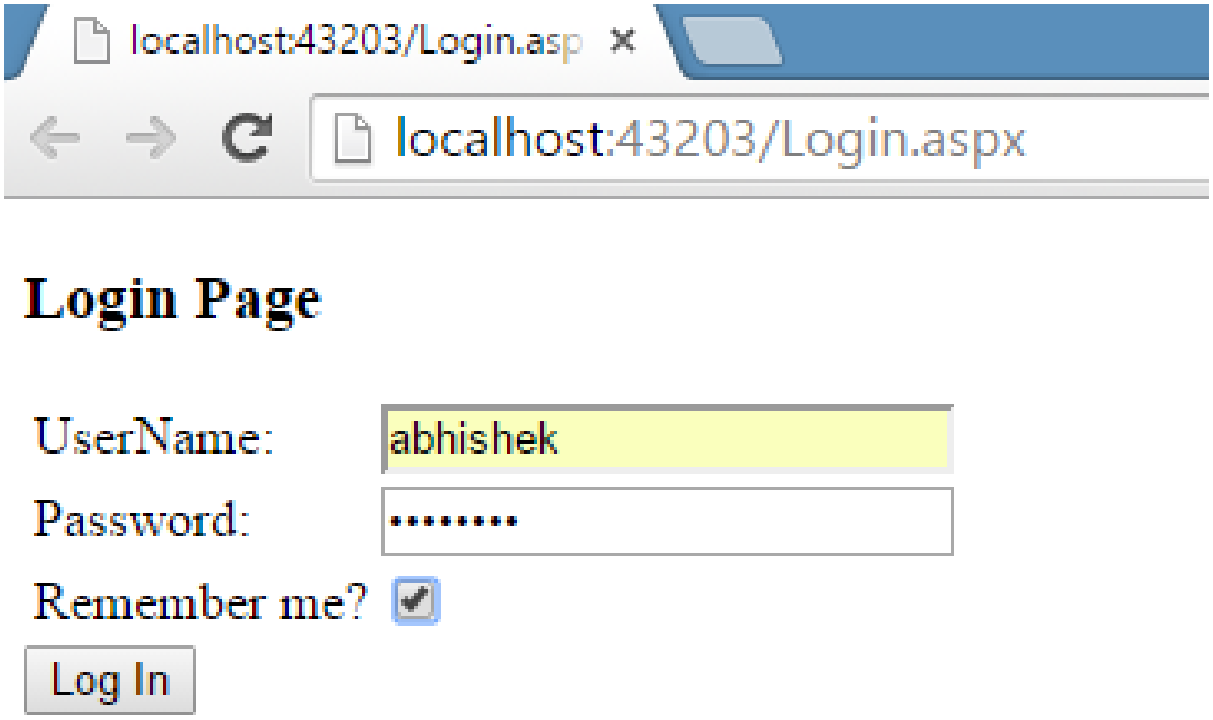


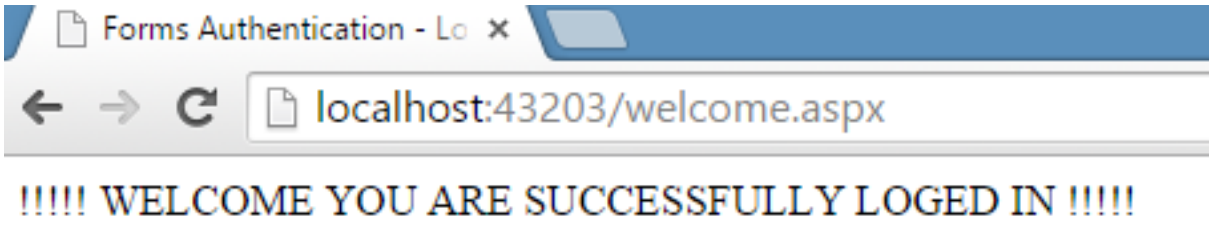
Here we enter the valid Username and Password which we assigned in our web.config file. If we enter any invalid user then it will throw an exception.



Step 8: By entering the valid username and password our application will successfully authenticate the user which will redirect to the next welcome page after successfully login.

Here we checked the **Remember me** check box which means let's say you logged in and close the browser. You again open the browser with welcome page then you can access the authorized page without login. That means our browser cookies are enabled which store the user data in text file until the browser is cleared.





Finally, valid user successfully logged in our web application using Forms Authentication in ASP.NET.

Practical No.14

Login Application

Step 1: Set up the Project and Database

Open Visual Studio 2012 and create a new ASP.NET WebForm Web Site or ASP.NET Web Application (.NET Framework) project using C#.

In Solution Explorer, right-click the project name, select Add, then Add New Item, and choose SQL Server Database (an .mdf file). Name it something like UserDB.mdf. This file will be added to the App_Data folder.

Open the Server Explorer (View > Server Explorer), and expand the data connection to your new database.

Right-click on the Tables folder and select Add New Table.

Define the table schema for your user information. For example, add columns like UserId (VARCHAR(50), Primary Key) and Password (VARCHAR(100)).

Save the table (e.g., as Users). Right-click the table and select Show Table Data to add a sample user account for testing.

Insert Sample User

1. Right-click **Users table**
2. Select **View Data**
3. Add sample record:

UserId	Password
admin	1234

Save and close.

Step 2: Design the Login Web Form

Right-click on your project in Solution Explorer, select Add, then Add New Item, and choose Web Form. Name it Login.aspx.

Design the Login.aspx page using standard ASP.NET controls.

Drag and drop two Label controls (for "Username" and "Password").

Drag and drop two TextBox controls. Set the TextMode property of the password TextBox to Password in the Properties window.

Drag and drop a Button control for logging in.

Drag and drop a Label control to display error or success messages.

Step 3: Configure the Connection String

Open your web.config file.

Add a <connectionStrings> section to define your database connection. The connection string for a LocalDB or SQL Server Express database file (mdf) will look like this:

xml

```
<connectionStrings>
  <add name="UserDBConn"
      connectionString="Data Source=(LocalDB)\v11.0;AttachDbFilename=|DataDirectory|\UserDB.mdf;Integrated Security=True"
      providerName="System.Data.SqlClient" />
</connectionStrings>
```

Step 4: Implement Login Logic in C#

Double-click the Login button on your Login.aspx page to create a Button1_Click event handler in the code-behind file (Login.aspx.cs).

In the Login.aspx.cs file, add the necessary namespaces at the top:

```
using System.Data.SqlClient;
using System.Configuration;
using System.Data;
```

Add the following code inside the Button1_Click event handler to connect to the database and validate user credentials.

csharp

```
protected void Button1_Click(object sender, EventArgs e)
{
  string conString = ConfigurationManager.ConnectionStrings["UserDBConn"].ConnectionString;
  using (SqlConnection con = new SqlConnection(conString))
  {
    // Use parameterized queries to prevent SQL injection
    string query = "SELECT COUNT(*) FROM Users WHERE UserId = @Username AND Password = @Password";
    SqlCommand cmd = new SqlCommand(query, con);
    cmd.Parameters.AddWithValue("@Username", TextBox1.Text);
    cmd.Parameters.AddWithValue("@Password", TextBox2.Text); // Passwords should be hashed in a real app
   con.Open();
    int count = (int)cmd.ExecuteScalar(); // Get the number of matching rows
```

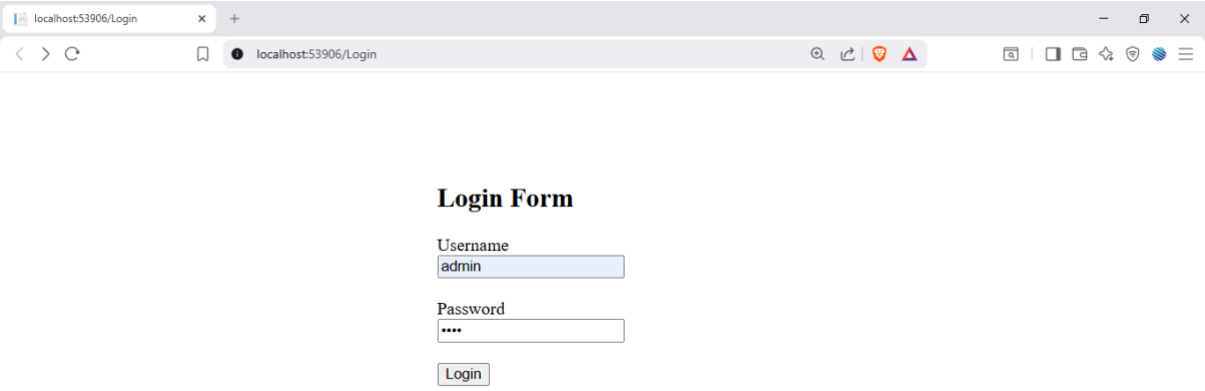
```
if (count > 0) {
    // Login successful - redirect to another page
    Response.Redirect("Default.aspx");    }
else    {
    // Login failed - display an error message
    Label1.Text = "Invalid Username or Password"; }}}
```

Step 5: Run the Application

Press F5 to run your application. Test the login functionality with the credentials you added to your database table.

For a real-world application, implement robust security measures like password hashing and Forms Authentication.

Output:



Practical No 15

Build the responsive page using bootstrap and implement AJAX for dynamic updates

Step 1: Create a Web Forms Project

Open Visual Studio 2012

File → New → Project

Select:

ASP.NET Web Forms Application

Click OK

Right click on project name i.e. AJAX EXAMPLE from solution explorer and add new web form give name is Webform1

Step 2: Add ScriptManager and UpdatePanel. Open the Webform1.aspx file and click on design tab at bottom side. Drag and drop ScriptManager and UpdatePanel from toolbox on webform1 inside <body>.

Step 3: Drag and drop the Labe1 and Button controls in UpdatePanel and change the Text Property of Label as “CLICK THE BUTTON” and Button is “UPDATE TIME”

Open Webform1.aspx and modify it like this:

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Default.aspx.cs"
Inherits="AjaxDemo.Default" %>

<!DOCTYPE html>

<html>

<head runat="server">

   <title>AJAX Demo</title>

</head>

<body>

   <form id="form1" runat="server">

      <asp:ScriptManager ID="ScriptManager1" runat="server"></asp:ScriptManager>

      <asp:UpdatePanel ID="UpdatePanel1" runat="server">

         <ContentTemplate>

            <asp:Label ID="Label1" runat="server" Text="Click the button"></asp:Label>

            <br /><br />

            <asp:Button ID="Button1" runat="server" Text="Update Time"
```

```
            OnClick="Button1_Click" />
        </ContentTemplate>
    </asp:UpdatePanel>
  </form>
</body>
</html>
```

Step 4: Double click on button in Webform1.aspx, it will open button1_click event code in Webform1.cs file –

Add the following code in Webform1.cs file –

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
namespace AJAX_EXAMPLE{
  public partial class WebForm1 : System.Web.UI.Page{
    protected void Page_Load(object sender, EventArgs e){}
    protected void Button1_Click(object sender, EventArgs e){
      Label1.Text = "Updated Time: " + DateTime.Now.ToString();}}}
```

<u>Output:</u>

When you click the button:

Only the UpdatePanel refreshes

No full page reload

AJAX works automatically

METHOD 2: Using jQuery AJAX + WebMethod (More Professional)

This gives better control and is closer to real-world applications.

Step 1: Add jQuery

Add this inside <head>:

```
<script src="https://code.jquery.com/jquery-1.8.3.min.js"></script>
```

(VS 2012 era compatible version)

Step 2: Add the button control from html toolbox and change the id attribute is “btnGetTime” and Value is “Get Server Time”. Add the span control manually in the code –

```
<form id="form1" runat="server">
  <div>
    <input type="button" id="btnGetTime" value="Get Server Time" />
    <br /><br />
    <span id="result"></span>
  </div>
</form>
```

Step 3: Add the javascript in your html page after the </form> tag.

```
<script type="text/javascript">
  $(document).ready(function () {
    $("#btnGetTime").click(function () {
      $.ajax({
        type: "POST",
        url: "Default.aspx/GetTime",
        contentType: "application/json; charset=utf-8",
        dataType: "json",
        success: function (response) {
          $("#result").html(response.d);  },
        error: function (error) {
          alert("Error: " + error.statusText);
        }  });  }); });
</script>
```

Step 3: Add WebMethod in Code Behind

Open WebForm1.aspx.cs:

```
using System;
using System.Web.Services;
namespace AjaxDemo{
   public partial class Default : System.Web.UI.Page{
      [WebMethod]
      public static string GetTime()   {
         return "Server Time: " + DateTime.Now.ToString();}}}
```

Output:

Button click sends AJAX request

Server returns data

Page does NOT reload

Cleaner and more scalable